Activation Functions in Neural Networks(Cont)

Sigmoid • tanh • ReLU • Leaky ReLU • PReLU

Sigmoid Activation Function

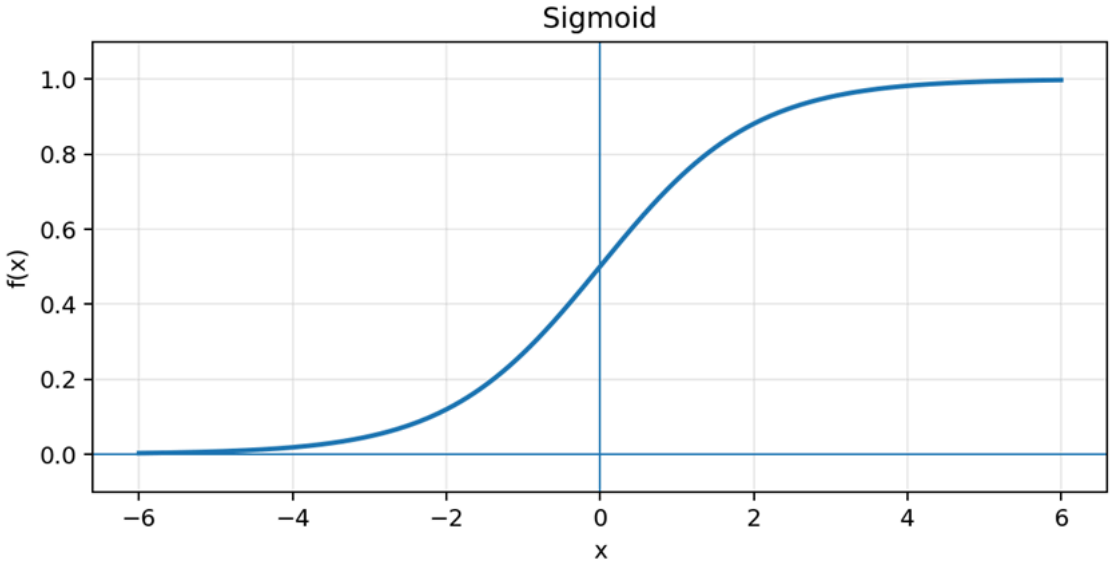
Sigmoid is a smooth S-shaped function that maps every real-valued input into (0, 1). Because its output resembles a probability, it is especially useful at the output of binary classifiers.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid expression

$$\sigma'(x) = \sigma(x)\,[1 - \sigma(x)]$$

Sigmoid derivative



Characteristics

- Range: (0, 1).
- Smooth and differentiable everywhere.
- Not zero-centered.
- Saturates near 0 and 1.
- Maximum derivative magnitude is 0.25.

Advantages

- Natural probability-like output.
- Smooth function.
- Useful for binary decisions and gates.

Disadvantages

- Vanishing gradients in saturation regions.
- Not zero-centered.
- Usually a poor default for deep hidden layers.
- Exponentiation is more expensive than simple piecewise functions.

Where It Is Used

- Binary classification output layers.

tanh Activation Function

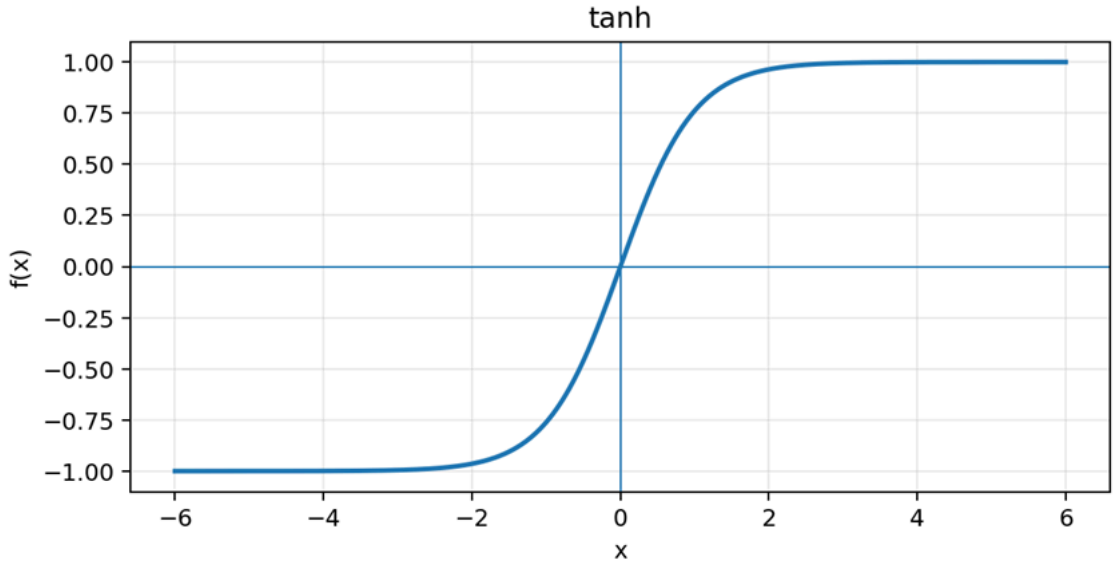
The hyperbolic tangent function maps inputs to (-1, 1). It is zero-centered, which is an important difference from sigmoid. It can represent positive and negative hidden states but still saturates.

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

tanh expression

$$\tanh'(x) = 1 - \tanh^2(x)$$

tanh derivative



Characteristics

- Range: (-1, 1).
- Zero-centered: tanh(0) = 0.
- Smooth and differentiable.
- The largest gradient occurs around zero.
- Saturates near -1 and +1.

Advantages

- Zero-centered outputs.
- Useful for bounded hidden states.
- Historically important in recurrent networks.

Disadvantages

- Vanishing gradients in saturation regions.
- Large input magnitudes can slow learning.

- Often replaced by ReLU-family activations in deep feed-forward networks.

Where It Is Used

RNN hidden-state transformations, LSTM/GRU state computations, and cases where bounded zero-centered representations are desirable.

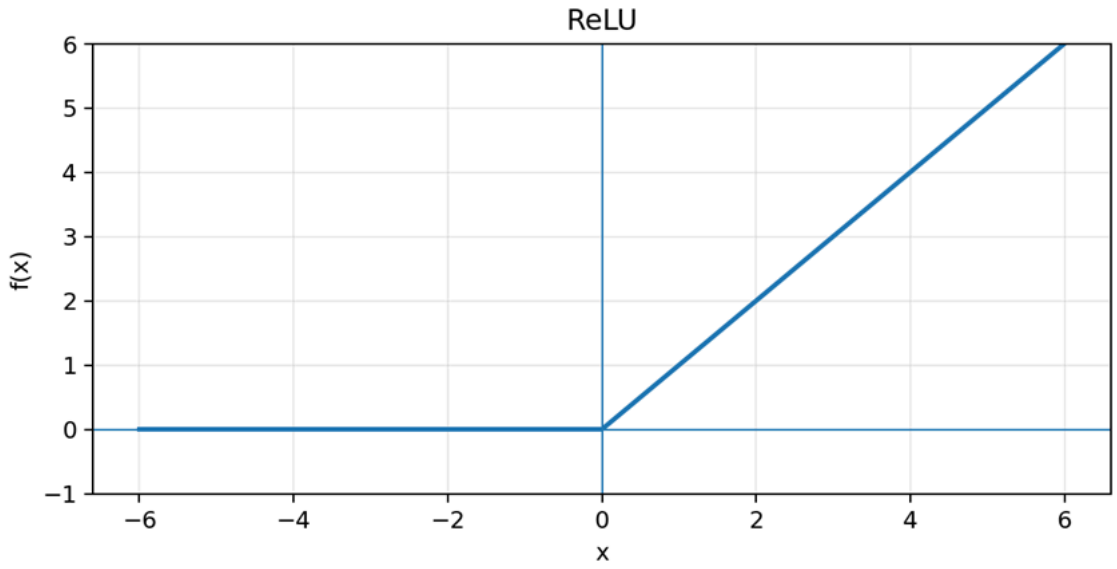
ReLU — Rectified Linear Unit

ReLU passes positive inputs unchanged and maps negative inputs to zero. It is one of the most widely used hidden-layer activations because it is simple and usually optimizes efficiently.

$$\mathrm{ReLU}(x) = \max(0, x)$$

ReLU expression

$$\mathrm{ReLU}'(x) = 0 \text{ for } x < 0, \qquad \mathrm{ReLU}'(x) = 1 \text{ for } x > 0$$



Characteristics

- Range: [0, ∞).
- Piecewise linear.
- Positive-side gradient is 1.
- Negative inputs produce exactly zero.
- Produces sparse activations.

Advantages

- Fast to compute.
- Strong baseline for deep hidden layers.
- No saturation on the positive side.
- Sparse activation can be useful.

Disadvantages

- Dying-ReLU problem: a neuron can become inactive when it stays on the negative side.
- The negative-side gradient is zero.
- Not zero-centered.
- Positive output is unbounded.

Where It Is Used

Hidden layers in CNNs and many feed-forward deep neural networks.

Leaky ReLU

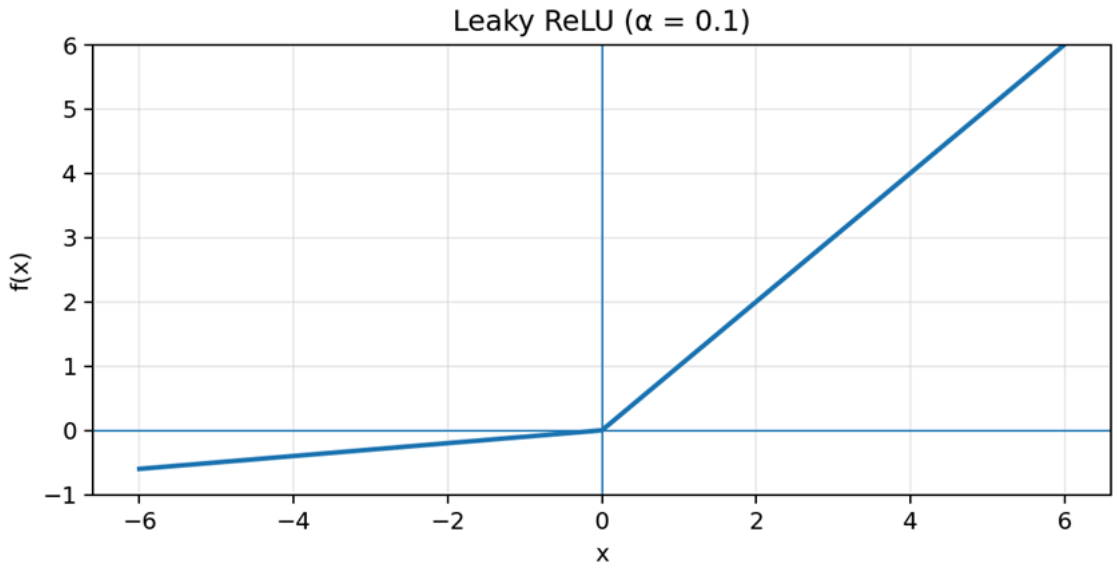
Leaky ReLU modifies ReLU by allowing a small, non-zero slope for negative inputs. Thus negative inputs still transmit some gradient.

$$\text{LeakyReLU}(x) = x \text{ for } x > 0, \qquad \text{LeakyReLU}(x) = \alpha x \text{ for } x \leq 0, \qquad \alpha > 0$$

Leaky ReLU expression

$$\text{LeakyReLU}'(x) = 1 \text{ for } x > 0, \qquad \text{LeakyReLU}'(x) = \alpha \text{ for } x < 0$$

Leaky ReLU derivative ($x \neq 0$)



Characteristics

- The positive side is identical to ReLU.
- The negative side has slope α.
- Reduces the dying-neuron problem.
- α is normally fixed as a hyperparameter.

Advantages

- Maintains gradient flow for negative inputs.
- Less prone to dying neurons than ReLU.
- Simple and inexpensive.

Disadvantages

- Requires choosing α.
- The chosen slope may not be optimal for every dataset.
- Still not zero-centered.

Where It Is Used

Hidden layers when standard ReLU causes too many inactive neurons; commonly considered in CNNs and other deep networks.

PReLU — Parametric ReLU

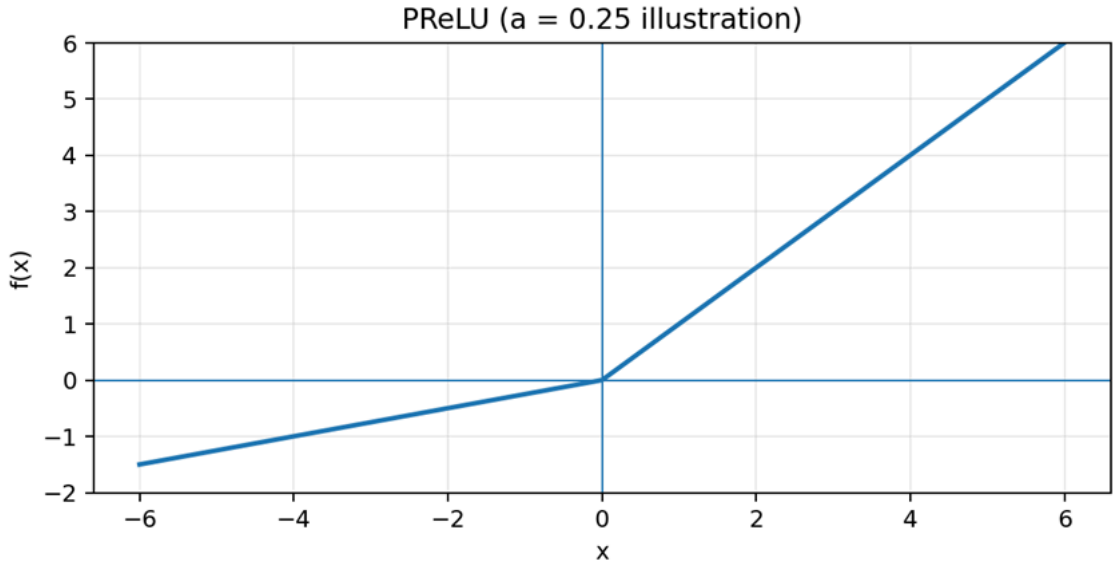
PReLU is similar to Leaky ReLU, but its negative slope is learned during training. This allows the model to adapt the negative branch to the data.

$$\mathrm{PReLU}(x) = x \text{ for } x > 0, \qquad \mathrm{PReLU}(x) = ax \text{ for } x \leq 0$$

PReLU expression

$$\mathrm{PReLU}'(x) = 1 \text{ for } x > 0, \qquad \mathrm{PReLU}'(x) = a \text{ for } x < 0$$

PReLU derivative (x ≠ 0)



Characteristics

- The positive branch is identical to ReLU.

- Negative slope is trainable.
- Can use learned parameters per channel or unit.
- Maintains a non-zero negative-side gradient.

Advantages

- Learn the negative slope from data.
- Reduces dying-neuron risk.
- More flexible than fixed-slope Leaky ReLU.

Disadvantages

- Adds trainable parameters.
- Can increase overfitting risk on small datasets.
- More complex than ReLU.
- Benefits depend on architecture and data.

Where It Is Used

Deep CNNs and experiments where an adaptive negative slope may improve optimization or representation learning.

ReLU vs Leaky ReLU vs PReLU

Property	ReLU	Leaky ReLU	PReLU
Negative input	0	αx	ax
Negative gradient	0	Fixed α	Learned a
Dying-neuron risk	Higher	Lower	Lower
Trainable slope	No	No	Yes
Core idea	Rectify	Leak a little	Learn the leak

Vanishing Gradient vs Dying ReLU

Vanishing gradients occur when derivatives become very small. Sigmoid and tanh are especially susceptible in their saturation regions. During backpropagation, repeated multiplication by small derivatives can make gradients in early layers extremely small.

Dying ReLU is different: the derivative is exactly zero for negative ReLU inputs. A neuron that remains negative can stop receiving useful gradient updates. Leaky ReLU and PReLU reduce this risk by keeping a non-zero negative-side slope.

Where to Use Each Function

Situation	Recommended	Reason
Binary classification output	Sigmoid	Output lies between 0 and 1.
Typical hidden layers	ReLU	Fast and strong baseline.
Dying-ReLU concern	Leaky ReLU	Non-zero negative gradient.
Adaptive negative slope desired	PReLU	Slope is learned.
Bounded zero-centered hidden state	tanh	Range is -1 to +1.

Comparison

Function	Range	Zero-Centered	Strength	Limitation	Typical Use
Sigmoid	(0, 1)	No	Probability-like output	Saturation	Binary output
tanh	(-1, 1)	Yes	Zero-centered	Saturation	RNNs / bounded states
ReLU	[0, ∞)	No	Fast and sparse	Dying neurons	Hidden layers
Leaky ReLU	(-∞, ∞)	No	Negative gradient	Fixed α	Hidden layers
PReLU	(-∞, ∞)	No	Learned negative slope	Extra parameter	CNNs

Summary

- Sigmoid is primarily an output activation for binary or independent probability-like predictions.
- tanh is zero-centered and useful for bounded hidden states, but it can saturate.
- ReLU is a strong default for many hidden layers and produces sparse activations.
- Leaky ReLU keeps a small negative gradient and reduces the dying-ReLU problem.
- PReLU learns the negative slope and adds flexibility at the cost of trainable parameters.
- Activation selection should consider the layer, task, optimization behavior, and empirical validation.